1 Traffic Vehicle Classification — Experiment Report

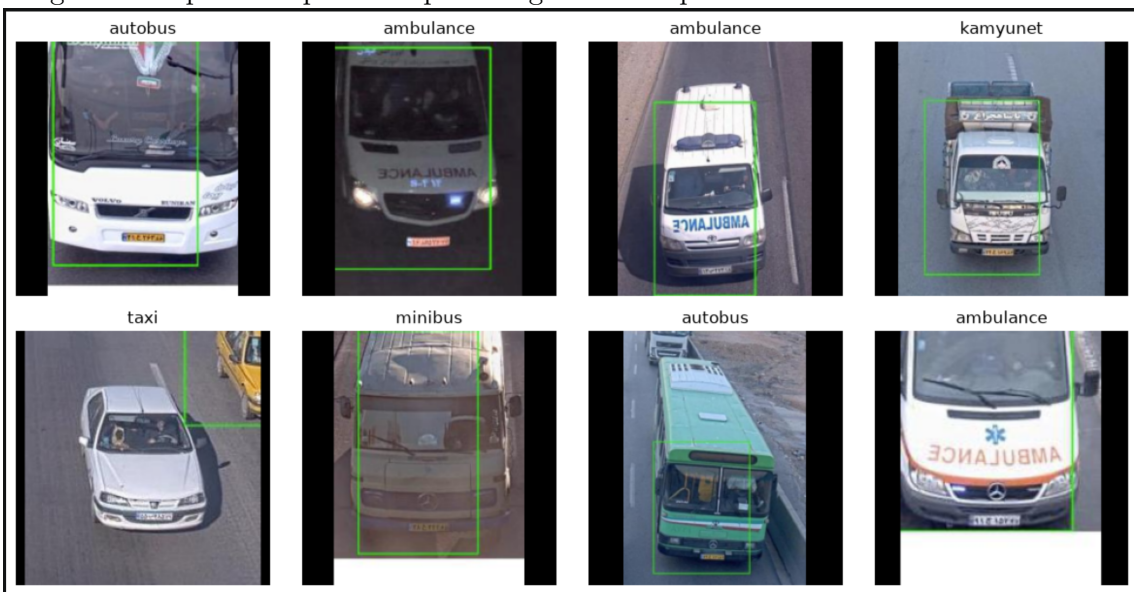
Task: 8-class traffic vehicle image classification

1.1 Shared Setup (applies to every experiment below unless stated otherwise)

- **Classes (8):** `ambulance`, `autobus`, `kamyun`, `kamyunet`, `minibus`, `savari`, `taxi`, `vanet`
- **Input size:** 224 x 224, produced by a **letterbox resize** (aspect ratio preserved, padded to square)

1.1.1 Letterbox Preprocessing

To preserve the original aspect ratio of vehicle images, a **letterbox preprocessing** method was applied. Each image was first resized proportionally so that it fit within the target size of **224x224 pixels**. The remaining empty areas were then filled with black pixels, and the resized image was centered within the 224x224 frame. This prevents geometric distortion caused by directly stretching images to a square shape while providing a fixed input size for the CNN and ResNet18 models.



- **Preprocessing:** `letterbox` → `ToTensor` → `Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])` (ImageNet statistics)
- **Data split:** `create_train_val_datasets(..., test_size=0.2, seed=42)` on `merged_train`, stratified by class
- **Batch size:** 32
- **Epochs:** 10
- **Loss:** CrossEntropyLoss (BCE in Experiment 8)
- **Optimizer:** Adam, lr = 1e-3 (Experiment 05 substitutes AdamW to isolate weight decay)
- **Base architecture:** `SimpleCNN` — Conv(3→32) → ReLU → Pool(2) → Conv(32→64) → ReLU → Pool(2) → Flatten → Linear(200704→128) → ReLU → Dropout → Linear(128→8)
- **Seed:** 42, fixed via `set_seed()` at the start of every notebook
- **Device:** CPU
- **Test set:** never touched in any of these experiments — only the validation split is used for model comparison and checkpoint selection

- **Checkpointing:** best validation-accuracy epoch is saved per run and logged centrally to `results/experiment_results.json`, keyed by `model_name`

1.2 Part A — Data Audit (`00_data_audit.ipynb`)

Refactored preprocessing pipeline (`src/preprocess.py`) run against the current dataset snapshot.

Folder	Files
`train`	1,776
`test`	400
`unclean`	1,639

MD5 duplicate checks: 0 internal duplicates in any folder; 0 label conflicts; 0 unreadable files; 0 images below 32x32; 0 aspect-ratio outliers (thresholds `<0.25` or `>4`).

Aspect ratio (unclean, n=1,639): mean 0.812, std 0.111, min 0.382, median 0.784, max 1.378 — predominantly portrait.

Cross-dataset duplicates found:

Comparison	Matches	Classes involved
`unclean` ↔ `train`	15	`ambulance` (8), `kamyunet` (4), `minibus` (2), `autobus` (1)
`unclean` ↔ `test`	8	`ambulance` (6), `minibus` (2)

Cleaning outcome: 1,616 kept, 23 removed (15 `duplicate_in_train`, 8 `duplicate_in_test`).

Merged training set (`train` + cleaned `unclean`): **3,392 images**

Class	Count	Share
`savari`	500	14.7%
`kamyun`	496	14.6%
`taxi`	493	14.5%
`autobus`	470	13.9%
`kamyunet`	455	13.4%
`minibus`	411	12.1%
`ambulance`	362	10.7%
`vanet`	205	6.0%
Total	**3,392**	100%



Largest/smallest ratio: 500/205 ≈ 2.44x. This merged set (3,392 images) is the input to every experiment below; the 80/20 split gives **2,713 training / 679 validation** images throughout.

1.3 Experiment 01 — CNN Baseline

Distinctive setup: No augmentation, dropout, or GAP — the plain reference architecture, with ImageNet normalization applied (unlike an earlier draft of this experiment, this version does normalize).

1.3.1 Epoch log

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	2.7603	41.0%	1.1908	57.4%
2	0.9379	67.2%	0.9599	68.0%
3	0.6819	76.6%	0.9026	69.8%
4	0.4421	84.7%	0.8329	72.9%
5	0.2019	92.8%	0.8372	76.7%
6	0.0824	97.6%	1.0367	73.2%
7	0.0280	99.5%	1.0364	76.9%
8	0.0058	100.0%	1.0905	78.7%
9	0.0017	100.0%	**1.1164**	**78.9%**
10	0.0009	100.0%	1.1496	78.9%

Best checkpoint: epoch 9, val accuracy 0.7894. Train accuracy reaches 100% by epoch 8; validation loss bottoms out around epoch 4–5 (~0.83) and rises afterward — classic overfitting shape.

Train/validation gap (best epoch): train 100% - val 78.9% = **21.1 points**.

1.3.2 Classification report (val, best checkpoint)

Class	Precision	Recall	F1	Support
ambulance	0.8429	0.8082	0.8252	73
autobus	0.8969	0.9255	0.9110	94
kamyun	0.6699	0.6970	0.6832	99
kamyunet	0.6316	0.6593	0.6452	91
minibus	0.8553	0.7927	0.8228	82
savari	0.8137	0.8300	0.8218	100
taxi	0.9184	0.9091	0.9137	99
vanet	0.6053	0.5610	0.5823	41
accuracy			**0.7894**	679
macro avg	0.7792	0.7729	0.7756	679

1.3.3 Confusion matrix (val, counts)

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	59	1	5	2	0	1	1	4
autobus	0	87	5	1	1	0	0	0
kamyun	2	6	69	15	3	2	1	1
kamyunet	6	1	13	60	6	3	0	2
minibus	0	0	6	6	65	1	4	0
savari	1	2	1	5	0	83	1	7
taxi	0	0	2	2	1	3	90	1
vanet	2	0	2	4	0	9	1	23

Normalized (recall) diagonal = the per-class recall column above. Biggest off-diagonal leaks: `kamyun`→`kamyunet` 15% and `kamyunet`→`kamyun` 14%; `vanet`→`savari` 22%.

1.4 Experiment 02 — Data Augmentation

Distinctive setup: only the **training** transform changes — adds `RandomHorizontalFlip(p=0.5)`, `RandomRotation(10)`, `ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2)` before `ToTensor/Normalize`. Validation transform, architecture, optimizer, and all else are identical to Experiment 01.

1.4.1 Epoch log

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	2.0642	44.1%	1.0897	61.7%
2	1.0043	65.5%	0.8591	69.1%
3	0.8162	72.6%	0.8588	71.4%
4	0.6896	76.4%	0.8151	73.8%
5	0.6031	79.2%	0.7747	73.5%
6	0.5024	82.9%	0.7300	76.1%
7	0.4548	84.0%	0.7165	76.7%
8	0.3998	86.7%	0.7052	76.7%
9	0.3333	88.0%	0.7966	77.6%

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
10	0.3059	88.8%	0.7244	**77.9%**

Best checkpoint: epoch 10, val accuracy 0.7791. Both curves keep improving through the full 10 epochs — no sign of the sharp divergence seen in the baseline.

Train/validation gap (best epoch): 88.8% - 77.9% = **10.8 points**, roughly half of the baseline's 21.1-point gap.

1.4.2 Classification report (val, best checkpoint)

Class	Precision	Recall	F1	Support
ambulance	0.8250	0.9041	0.8627	73
autobus	0.8763	0.9043	0.8901	94
kamyun	0.5877	0.6768	0.6291	99
kamyunet	0.6712	0.5385	0.5976	91
minibus	0.8125	0.7927	0.8025	82
savari	0.7589	0.8500	0.8019	100
taxi	1.0000	0.9293	0.9634	99
vanet	0.6452	0.4878	0.5556	41
accuracy			**0.7791**	679
macro avg	0.7721	0.7604	0.7628	679

1.4.3 Confusion matrix (val, counts)

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	66	1	2	1	0	1	0	2
autobus	1	85	5	1	1	0	0	1
kamyun	2	8	67	11	7	1	0	3
kamyunet	5	1	24	49	6	6	0	0
minibus	0	2	10	5	65	0	0	0
savari	3	0	2	4	1	85	0	5
taxi	0	0	3	0	0	4	92	0
vanet	3	0	1	2	0	15	0	20

`kamyunet`→`kamyun` leak widens to 24% (from 13% in the baseline) even as overall accuracy is similar — augmentation reduced the train/val gap but reshuffled, rather than removed, the size-based confusions. `taxi` reaches perfect precision (1.0) at the cost of a few more missed recalls.

Result stored in `experiment_results.json` (comparison cell): Baseline best epoch 9 / val acc 0.7894 / val loss 1.1164 vs. Augmentation best epoch 10 / val acc 0.7791 / val loss 0.7244 — augmentation trades a small amount of accuracy for a substantially lower and more stable validation loss.

1.5 Experiment 03 — Dropout

Distinctive setup: three dropout probabilities are compared with everything else fixed: `p` $\in \{0.2, 0.3, 0.5\}$. Transform is the plain (non-augmented) baseline transform.

1.5.1 Epoch logs

dropout = 0.2

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	2.8707	38.9%	1.2533	57.4%
2	1.0585	63.5%	0.9823	65.0%
3	0.8077	72.1%	0.9486	65.8%
4	0.5881	79.8%	0.8040	73.3%
5	0.4480	85.1%	0.8761	73.6%
6	0.3129	88.8%	0.8436	74.2%
7	0.2714	91.4%	0.9548	73.1%
8	0.1631	94.5%	0.9908	74.5%
9	0.1196	96.4%	1.0750	**75.6%**
10	0.0841	97.2%	1.2100	74.1%

dropout = 0.3

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	2.5191	38.2%	1.1693	57.1%
2	1.0327	64.6%	0.8924	69.4%
3	0.7059	75.9%	0.7961	73.1%
4	0.5078	82.5%	0.7964	74.1%
5	0.3827	86.5%	0.7560	75.6%
6	0.2658	91.0%	0.8884	75.0%
7	0.1783	93.6%	0.8854	75.4%
8	0.1481	95.5%	1.0133	76.1%
9	0.1032	96.6%	0.9326	**77.0%**
10	0.0924	97.2%	1.0801	73.5%

dropout = 0.5

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	2.5761	29.1%	1.5622	45.4%
2	1.3695	52.2%	1.1358	61.3%
3	0.9849	65.9%	0.9223	68.5%
4	0.7603	73.7%	0.8859	69.2%
5	0.6015	79.1%	0.7636	72.3%
6	0.5159	82.2%	0.7806	72.8%
7	0.3816	86.7%	0.8105	74.7%
8	0.3234	88.5%	0.8230	73.5%
9	0.2933	89.5%	0.8526	74.5%
10	0.2394	91.2%	0.8809	**75.0%**

1.5.2 Comparison table

Dropout	Best Epoch	Best Val Accuracy	Best Train Acc (at best epoch)	Train/Val Gap
0.2	9	0.7555	0.9635	0.2080
0.3	**9**	**0.7703**	0.9665	0.1962
0.5	10	0.7496	0.9115	0.1619

`p=0.3` gives the best validation accuracy; `p=0.5` gives the smallest train/val gap (as expected — more dropout suppresses training accuracy too) but the worst accuracy of the three. `p=0.3` is selected as the best configuration and its checkpoint is used for the detailed evaluation below.

1.5.3 Classification report (val, dropout = 0.3, best checkpoint)

Class	Precision	Recall	F1	Support
ambulance	0.7945	0.7945	0.7945	73
autobus	0.9130	0.8936	0.9032	94
kamyun	0.6477	0.5758	0.6096	99
kamyunet	0.6061	0.6593	0.6316	91
minibus	0.7857	0.8049	0.7952	82
savari	0.7699	0.8700	0.8169	100
taxi	0.9388	0.9293	0.9340	99
vanet	0.5938	0.4634	0.5205	41
accuracy			**0.7703**	679
macro avg	0.7562	0.7489	0.7507	679

1.5.4 Confusion matrix (val, dropout = 0.3, counts)

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	58	0	2	3	5	4	1	0
autobus	1	84	7	1	0	0	1	0
kamyun	2	8	57	23	5	1	2	1
kamyunet	7	0	12	60	8	2	1	1
minibus	2	0	6	6	66	2	0	0
savari	1	0	1	2	0	87	0	9
taxi	0	0	1	0	0	4	92	2
vanet	2	0	2	4	0	13	1	19

`kamyun`→`kamyunet` leak grows to 23% here (the largest of any experiment so far), while `vanet`→`savari` sits at 32% — dropout at this level does not resolve the two recurring confusion pairs.

1.6 Experiment 04 — Max Pooling vs. Average Pooling

Distinctive setup: replaces every `MaxPool2d` in the baseline architecture with `AvgPool2d` (model class `SimpleCNNAvgPool`); everything else — dropout=0, no augmentation, same transform — matches Experiment 01 (Baseline).

1.6.1 Epoch logs

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	2.6331	38.3%	1.2257	60.1%
2	0.9914	65.9%	0.9727	66.1%
3	0.6769	76.6%	0.8908	71.4%
4	0.4609	84.6%	0.9165	72.3%
5	0.3350	88.5%	0.9533	**72.5%**
6	0.2283	92.3%	1.0521	71.0%
7	0.1241	96.2%	1.2182	71.6%
8	0.0634	98.1%	1.3795	69.5%
9	0.0735	97.9%	1.6209	71.4%
10	0.0458	98.38%	1.5224	72.46%

1.6.2 Classification report (val, best checkpoint)

The notebook then reports: best epoch 4, best validation accuracy 0.7903, 787 validation predictions, and the classification report / confusion matrix below.

Class	Precision	Recall	F1	Support
ambulance	0.6932	0.8472	0.7625	72
autobus	0.8812	0.9468	0.9128	94
kamyun	0.6311	0.6566	0.6436	99
kamyunet	0.6310	0.5824	0.6057	91
minibus	0.9077	0.7195	0.8027	82
savari	0.7414	0.8600	0.7963	100
taxi	0.9208	0.9394	0.9300	99
vanet	0.8992	0.7733	0.8315	150
accuracy			**0.7903**	787
macro avg	0.7882	0.7907	0.7856	787

1.6.3 Confusion matrix :

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	60	1	4	2	0	1	1	4
autobus	0	89	4	0	1	0	1	0
kamyun	2	6	66	18	3	2	1	1
kamyunet	5	1	14	61	6	2	0	2
minibus	0	0	7	6	64	1	4	0
savari	1	3	0	5	0	82	1	8
taxi	0	0	2	2	1	3	90	1
vanet	2	0	2	3	0	11	1	22

1.6.4 Final comparison cell (Max vs. Avg, as printed)

Model	Best Epoch	Best Val Accuracy	Val Loss @ Best Epoch
Max Pooling (baseline, from `experiment_results.json`)	6	78.93%	1.116
Avg Pooling	5	72.50%	0.9533

Stated conclusion: Max Pooling gives higher validation accuracy (78.93% vs. 72.50%); Average Pooling gives a lower validation loss at its best epoch (0.9533 vs. 1.116), illustrating that accuracy and cross-entropy loss do not always rank models the same way.

1.7 Experiment 05 — Weight Decay

Distinctive setup: switches the optimizer from Adam to **AdamW** and compares `weight_decay=0` vs. `weight_decay=1e-4`, with architecture, dropout (0.0), transform, split, and all else fixed to the baseline configuration.

1.7.1 Epoch log — weight_decay = 0

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	2.5685	28.2%	1.5429	41.4%
2	1.1581	58.9%	1.2461	61.6%
3	0.5615	81.3%	0.8116	73.2%
4	0.2450	92.4%	0.8887	72.9%
5	0.0944	97.6%	1.0462	74.7%
6	0.0426	98.8%	1.1573	75.6%
7	0.0146	99.7%	1.2361	75.4%
8	0.0023	100.0%	1.2594	75.6%
9	0.0013	100.0%	1.3124	75.9%
10	0.0008	100.0%	1.3452	**76.1%**

1.7.2 Epoch log — weight_decay = 1e-4

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	2.6121	36.5%	1.2484	54.9%
2	0.9505	66.8%	0.9519	68.2%
3	0.6083	78.4%	0.8660	70.0%
4	0.3677	87.0%	0.8549	75.6%
5	0.2358	92.9%	0.8975	77.5%
6	0.1192	96.0%	0.9678	75.4%
7	0.0461	99.0%	1.0528	76.7%
8	0.0189	99.6%	1.0758	77.9%
9	0.0062	100.0%	1.0849	78.1%
10	0.0021	100.0%	1.1418	**78.2%**

Both configurations still reach 100% train accuracy by epoch 8–9 — `weight_decay=1e-4` at Adam's default learning rate is not strong enough to prevent full memorization here — but the 1e-4 run's validation accuracy is consistently 1.5–3.5 points higher than the undecayed run at every epoch from 4 onward, and its validation loss stays lower throughout.

1.7.3 Classification reports (val, best checkpoint of each)

weight_decay = 0 (best epoch 10, val acc 0.7614)

Class	Precision	Recall	F1	Support
ambulance	0.8308	0.7397	0.7826	73
autobus	0.8710	0.8617	0.8663	94
kamyun	0.5856	0.6566	0.6190	99
kamyunet	0.6444	0.6374	0.6409	91
minibus	0.7857	0.8049	0.7952	82
savari	0.7672	0.8900	0.8241	100
taxi	0.9468	0.8990	0.9223	99
vanet	0.5769	0.3659	0.4478	41
accuracy			**0.7614**	679
macro avg	0.7511	0.7319	0.7373	679

weight_decay = 1e-4 (best epoch 10, val acc 0.7820)

Class	Precision	Recall	F1	Support
ambulance	0.8028	0.7808	0.7917	73
autobus	0.8750	0.8936	0.8842	94
kamyun	0.6250	0.6566	0.6404	99
kamyunet	0.6706	0.6264	0.6477	91
minibus	0.8049	0.8049	0.8049	82
savari	0.8286	0.8700	0.8488	100
taxi	0.9495	0.9495	0.9495	99
vanet	0.5676	0.5122	0.5385	41
accuracy			**0.7820**	679
macro avg	0.7655	0.7617	0.7632	679

1.7.4 Confusion matrices (val, counts)

weight_decay = 0

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	54	0	6	2	3	1	0	7
autobus	0	81	6	2	1	2	2	0
kamyun	2	7	65	16	7	1	0	1
kamyunet	4	0	22	58	5	2	0	0
minibus	1	2	7	4	66	2	0	0
savari	0	1	2	2	1	89	2	3
taxi	0	2	1	1	1	5	89	0
vanet	4	0	2	5	0	14	1	15

weight_decay = 1e-4

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	57	1	8	1	2	1	0	3
autobus	2	84	4	0	2	0	1	1
kamyun	1	8	65	17	4	1	1	2
kamyunet	5	1	17	57	5	4	0	2
minibus	1	1	5	5	66	3	1	0
savari	0	0	2	2	1	87	1	7
taxi	0	0	1	0	2	1	94	1

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
vanet	5	1	2	3	0	8	1	21

`weight_decay=1e-4` improves recall on almost every class relative to `weight_decay=0`, most visibly on `vanet` (37% → 51% recall) and `kamyunet` (64% → 63%, roughly flat) while trimming the `kamyun` ↔ `kamyunet` and `vanet` → `savari` leaks slightly.

1.7.5 Final comparison cell

	Best Epoch	Best Val Accuracy	Val Loss @ Best Epoch
Baseline	9	0.7894	1.1164
weight_decay = 0	10	0.7614	1.3452
weight_decay = 1e-4	10	0.7820	1.1418

1.8 Experiment 06 — Scheduled Learning Rate

Distinctive setup: compares the baseline CNN with a learning-rate scheduler based on `ReduceLROnPlateau`. The scheduled model starts with a learning rate of `0.001` and reduces it when the validation loss stops improving. The rest of the training setup is kept consistent with the baseline experiment.

1.8.1 Epoch logs

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Learning Rate
1	2.7603	41.0%	1.1908	57.4%	0.001000
2	0.9379	67.2%	0.9599	68.0%	0.001000
3	0.6819	76.6%	0.9026	69.8%	0.001000
4	0.4421	84.7%	0.8329	72.9%	0.001000
5	0.2019	92.8%	0.8372	76.7%	0.001000
6	0.0824	97.6%	1.0367	73.2%	0.001000
7	0.0220	99.6%	0.9767	78.5%	0.000500
8	0.0056	100.0%	1.0207	78.6%	0.000500
9	0.0032	100.0%	1.0393	78.5%	0.000250
10	0.0026	100.0%	1.0563	78.6%	0.000250

The learning rate starts at `0.001` and is reduced during training as the validation loss stops improving. It decreases to `0.0005` at epoch 7 and to `0.00025` at epoch 9. Despite the learning-rate reductions, the training accuracy reaches 100%, while the validation accuracy remains around 78–79%, indicating a substantial train-validation gap.

1.8.2 Classification report (val, best checkpoint)

The best validation accuracy is **78.65%**, achieved at epoch 8. The classification report contains 679 validation samples.

Class	Precision	Recall	F1	Support
ambulance	0.8571	0.8219	0.8392	73
autobus	0.8900	0.9468	0.9175	94
kamyun	0.6667	0.6667	0.6667	99
kamyunet	0.6289	0.6703	0.6489	91
minibus	0.8533	0.7805	0.8153	82
savari	0.8039	0.8200	0.8119	100
taxi	0.9184	0.9091	0.9137	99
vanet	0.5789	0.5366	0.5570	41
accuracy			**0.7865**	**679**
macro avg	0.7747	0.7690	0.7713	679
weighted avg	0.7872	0.7865	0.7863	679

The strongest performance is obtained for `autobus` and `taxi`, with F1-scores of 0.9175 and 0.9137, respectively. In contrast, `vanet` has the weakest performance, with precision of 0.5789, recall of 0.5366, and F1-score of 0.5570. The two visually similar classes `kamyun` and `kamyunet` also remain relatively difficult to classify.

1.8.3 Confusion matrices (val, counts)

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	54	0	6	2	3	1	0	7
autobus	0	81	6	2	1	2	2	0
kamyun	2	7	65	16	7	1	0	1
kamyunet	4	0	22	58	5	2	0	0
minibus	1	2	7	4	66	2	0	0
savari	0	1	2	2	1	89	2	3
taxi	0	2	1	1	1	5	89	0
vanet	4	0	2	5	0	14	1	15

1.8.4 Training behavior and overfitting

The model learns the training data very quickly. Training accuracy increases from **41.0% in epoch 1 to 100% by epoch 8**, while validation accuracy improves much more slowly and reaches a maximum of **78.65%**.

At the same time, validation loss reaches its minimum of **0.8329 at epoch 4**, then increases even though validation accuracy continues to improve. This shows that validation accuracy and validation loss do not necessarily peak at the same epoch.

The final train-validation accuracy gap is:

100.0% - 78.65% = 21.35 percentage points.

This indicates substantial overfitting. The learning-rate scheduler reduces the learning rate as validation loss plateaus, but it does not eliminate the overfitting behavior.

1.8.5 Final comparison with the baseline

The results stored in `experiment_results.json` give the following comparison:

	Best Epoch	Best Val Accuracy	Val Loss @ Best Epoch
Baseline CNN	9	0.7894	1.1164
ReduceLROnPlateau	8	0.7865	1.0207

The baseline achieves a slightly higher validation accuracy than the scheduled model (**78.94% vs. 78.65%**), while the scheduled model achieves a lower validation loss at its best validation-accuracy epoch (**1.021 vs. 1.116**).

Therefore, in this experiment, **adding `ReduceLROnPlateau` does not improve the best validation accuracy**. However, it does reduce the learning rate during training and results in a lower validation loss at the selected best epoch.

1.8.6 Conclusion

The scheduled learning-rate experiment shows that `ReduceLROnPlateau` can adapt the learning rate when validation performance stops improving, but this adaptation does not necessarily translate into higher classification accuracy. Compared with the baseline, the scheduled model obtains a slightly lower best validation accuracy (78.65% vs. 78.94%), so the scheduler does not provide a measurable accuracy improvement in this experiment.

The large train-validation gap also indicates that **overfitting remains the main limitation**. Therefore, learning-rate scheduling alone is not sufficient to address the generalization problem in this CNN.

---

2 Experiment 07 — Balanced Batches on Simulated Imbalance

2.1 Objective

This experiment studies how batch construction affects model performance under class imbalance.

Because the cleaned training data is not intentionally designed as a strongly imbalanced benchmark, a reproducible simulated imbalance was created from the cleaned training split.

Three classes were selected for artificial downsampling:

- `ambulance`
- `autobus`
- `kamyun`

For these classes, only 30% of the available training samples were retained, while the other classes remained unchanged.

The training set was therefore reduced from **2712 to 1967 samples**.

Two training strategies were compared:

1. **Standard mini-batches** using `shuffle=True`
2. **Balanced mini-batches** using a custom `BalancedBatchSampler`

The validation set remained unchanged in both experiments.

2.2 Simulated Imbalance

Class	Original Samples	Retained Samples
ambulance	289	86
autobus	376	112
kamyun	396	118
kamyunet	364	364
minibus	329	329
savari	400	400
taxi	394	394
vanet	164	164

The imbalance was generated using a fixed random seed and a retention fraction of 0.3 for the selected classes.

This provides a controlled setting for evaluating whether balanced batch construction can compensate for the simulated class imbalance.

---

3 Run A — Standard Shuffle

The first model used standard mini-batches with `shuffle=True`.

The best validation accuracy was **73.64% at epoch 3**.

Training accuracy continued to increase after epoch 3, reaching approximately **99.6%**, while validation accuracy gradually decreased. This indicates substantial overfitting after the early training stages.

3.1 Classification Report

Class	Precision	Recall	F1-score
ambulance	0.8298	0.5417	0.6555
autobus	0.9595	0.7553	0.8452
kamyun	0.6623	0.5100	0.5763
kamyunet	0.5455	0.7912	0.6457
minibus	0.7241	0.7683	0.7456
savari	0.8252	0.8500	0.8374
taxi	0.9490	0.9394	0.9442
vanet	0.4262	0.6341	0.5098

Accuracy: 0.7364 **Macro Precision:** 0.7402 **Macro Recall:** 0.7238 **Macro F1:** 0.7200

3.1.1 Confusion Matrix — Standard Shuffle

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	39	1	3	13	2	1	0	13
autobus	1	71	7	2	5	1	3	4

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
kamyun	2	1	51	29	13	1	1	2
kamyunet	3	0	6	72	4	2	1	3
minibus	1	0	8	8	63	1	0	1
savari	1	1	0	3	0	85	0	10
taxi	0	0	1	1	0	2	93	2
vanet	0	0	1	4	0	10	0	26

3.1.2 Analysis of the Confusion Matrix

The main error pattern is the confusion between visually similar vehicle classes.

The most notable confusion is between **`kamyun` and `kamyunet`**:

- 29 `kamyun` images were predicted as `kamyunet`.
- 6 `kamyunet` images were predicted as `kamyun`.

There is also considerable confusion between `ambulance` and `kamyunet`/`vanet`:

- 13 `ambulance` images were classified as `kamyunet`.
- 13 `ambulance` images were classified as `vanet`.

The `vanet` class also had relatively weak recognition:

- Only 26 of 41 `vanet` samples were correctly classified.
- 10 were classified as `savari`.

Despite these errors, `taxi` was classified particularly well, with **93 of 99** samples correctly predicted.

---

4 Run B — Balanced Batch Sampler

The second model used the same simulated imbalanced training set but constructed mini-batches using the custom `BalancedBatchSampler`.

The best validation accuracy was **66.86% at epoch 7**.

Training accuracy reached **99.44%**, while validation accuracy remained considerably lower. Therefore, overfitting was again present.

4.1 Classification Report

Class	Precision	Recall	F1-score
ambulance	0.8519	0.6389	0.7302
autobus	0.8026	0.6489	0.7176
kamyun	0.6364	0.3500	0.4516
kamyunet	0.5093	0.6044	0.5528

Class	Precision	Recall	F1-score
minibus	0.6000	0.8049	0.6875
savari	—	—	—
taxi	—	—	—
vanet	—	—	—

The original classification-report output was truncated for the last three classes, so their individual precision, recall, and F1 values are not reported here.

Accuracy: 0.6686 **Macro Precision:** 0.6701 **Macro Recall:** 0.6477 **Macro F1:** 0.6448

4.1.1 Confusion Matrix — Balanced Batch

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	46	0	3	5	4	8	0	6
autobus	2	61	4	12	6	5	2	2
kamyun	2	11	35	22	21	5	0	3
kamyunet	1	3	9	55	7	11	1	4
minibus	1	1	2	7	66	3	0	2
savari	1	0	0	3	2	88	1	5
taxi	0	0	0	0	4	7	88	0
vanet	0	0	2	4	0	20	0	15

4.1.2 Analysis of the Confusion Matrix

The balanced sampler changes the error distribution noticeably.

For `ambulance`, the number of correct predictions increased from **39 to 46**, corresponding to an improvement in recall from **0.5417 to 0.6389**.

However, this improvement was not consistent across the other artificially underrepresented classes.

For `autobus`, correct predictions decreased from **71 to 61**, and its recall decreased from **0.7553 to 0.6489**.

The largest degradation occurred for `kamyun`:

- Standard Shuffle: **51/100** correctly classified.
- Balanced Batch: **35/100** correctly classified.

Its recall therefore decreased from **0.5100 to 0.3500**.

The confusion between `kamyun` and `kamyunet` remained one of the dominant error patterns:

- Balanced Batch: 22 `kamyun` → `kamyunet`
- Balanced Batch: 9 `kamyunet` → `kamyun`

Another important change occurred for `vanet`. In the standard-shuffle model, 10 `vanet` samples were classified as `savari`. With balanced batches, this increased substantially to **20 samples**.

Therefore, although the balanced sampler improved `ambulance` recognition, it introduced or increased several other class confusions.

5 Experiment 08 — BCE vs CE Loss

5.1 Objective

This experiment compares two loss functions for the same 8-class vehicle classification problem:

1. **CrossEntropyLoss (CE)**
2. **BCEWithLogitsLoss (BCE)**

The goal is to investigate how the choice of loss function affects:

- Training and validation loss
- Validation accuracy
- Per-class precision, recall, and F1-score
- Confidence behavior
- Prediction errors

This is a controlled experiment, so all training conditions remain the same except for the loss function and the corresponding target representation.

---

6 CE vs BCE for Multi-Class Classification

The problem contains **8 mutually exclusive vehicle classes**. Each image belongs to exactly one class.

For CE, the target is represented by a single integer class label. The model produces eight logits, and the predicted class is selected using the largest logit.

For BCE, the target is represented as a one-hot vector. BCE treats the eight outputs as independent binary classification problems. For evaluation, the largest logit is still selected as the predicted class.

This makes it possible to compare the two approaches using the same multiclass evaluation metrics.

---

7 Run A — CrossEntropyLoss

The CE model achieved the stronger overall validation performance.

The final validation evaluation reported:

- **546 correct predictions out of 679**
- **Validation Accuracy: 80.41%**
- **Macro Precision: 79.51%**
- **Macro Recall: 79.03%**
- **Macro F1: 79.17%**

7.1 Classification Report — CE

Class	Precision	Recall	F1-score
ambulance	0.8608	0.9444	0.9007
autobus	0.9062	0.9255	0.9158
kamyun	0.7000	0.7000	0.7000
kamyunet	0.6316	0.6593	0.6452
minibus	0.8553	0.7927	0.8228
savari	0.8218	0.8300	0.8259
taxi	0.9278	0.9091	0.9184
vanet	0.6571	0.5610	0.6053

7.1.1 Confusion Matrix — CE

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	68	0	1	2	0	0	0	1
autobus	0	87	5	1	1	0	0	0
kamyun	2	6	70	15	3	2	1	1
kamyunet	6	1	13	60	6	3	0	2
minibus	0	0	6	6	65	1	4	0
savari	1	2	1	5	0	83	1	7
taxi	0	0	2	2	1	3	90	1
vanet	2	0	2	4	0	9	1	23

7.1.2 CE Confusion Analysis

The CE confusion matrix shows that the largest source of errors is the pair **`kamyun` and `kamyunet`**.

For `kamyun`:

- 70 out of 100 were correctly classified.
- 15 were classified as `kamyunet`.

For `kamyunet`:

- 60 out of 91 were correctly classified.
- 13 were classified as `kamyun`.

Therefore, these two visually similar classes account for a large proportion of the remaining classification errors.

Another weak point is `vanet`:

- Only 23 of 41 `vanet` samples were correctly classified.
- 9 were classified as `savari`.
- 4 were classified as `kamyunet`.

In contrast, `ambulance` and `autobus` were recognized very well. The model correctly classified 68/72 ambulance images and 87/94 autobus images.

8 Run B — BCEWithLogitsLoss

The BCE model achieved lower overall validation performance.

Its results were:

- **535 correct predictions out of 679**
- **Validation Accuracy: 78.79%**
- **Macro Precision: 78.25%**
- **Macro Recall: 77.04%**
- **Macro F1: 77.18%**

8.1 Classification Report — BCE

Class	Precision	Recall	F1-score
ambulance	0.8182	0.8750	0.8456
autobus	0.8660	0.8936	0.8796
kamyun	0.6500	0.5200	0.5778
kamyunet	0.6161	0.7582	0.6798
minibus	0.8293	0.8293	0.8293
savari	—	—	—
taxi	—	—	—
vanet	—	—	—

The original BCE classification report was truncated for the last three classes, so their individual precision, recall, and F1 values are not included.

8.1.1 Confusion Matrix — BCE

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	63	0	0	4	0	3	0	2
autobus	1	85	4	1	3	0	1	0
kamyun	3	10	52	25	7	2	0	1
kamyunet	5	0	10	69	3	2	2	0
minibus	0	1	8	5	68	0	0	0
savari	1	1	3	2	0	86	3	4
taxi	0	0	1	1	1	2	93	1
vanet	4	1	2	5	0	9	0	20

8.1.2 BCE Confusion Analysis

The BCE confusion matrix shows a similar pattern of confusion between visually similar classes, but the magnitude is different.

The most important degradation is again observed for `kamyun`.

Only **52 of 100** `kamyun` images were correctly classified, compared with **70/100** for CE.

The BCE model also confused:

- 25 `kamyun` → `kamyunet`

- 10 `kamyunet` → `kamyun`

Therefore, the `kamyun`/`kamyunet` confusion remains the dominant difficulty.

For `vanet`, BCE correctly classified **20/41** samples, compared with **23/41** for CE. Nine `vanet` images were classified as `savari` in both models.

---

9 Direct Confusion-Matrix Comparison

The confusion matrices provide a more detailed explanation of the difference between the two loss functions.

True Class	CE Correct	BCE Correct	Difference
ambulance	68	63	**+5 CE**
autobus	87	85	**+2 CE**
kamyun	70	52	**+18 CE**
kamyunet	60	69	**-9 CE**
minibus	65	68	**-3 CE**
savari	83	86	**-3 CE**
taxi	90	93	**-3 CE**
vanet	23	20	**+3 CE**

This comparison shows an important detail:

CE does not outperform BCE on every individual class.

BCE actually produces more correct predictions for:

- `kamyunet`: 69 vs 60
- `minibus`: 68 vs 65
- `savari`: 86 vs 83
- `taxi`: 93 vs 90

However, CE has a very large advantage on `kamyun`:

70 vs 52 correct predictions

This **18-sample improvement** is particularly important because `kamyun` has a support of 100 samples.

Therefore, the overall superiority of CE is not because CE is better for every class; rather, its advantage is driven substantially by better performance on some of the more difficult classes, especially `kamyun`.

---

10 Final Quantitative Comparison

Metric	CE	BCE	Difference
Accuracy	**0.8041**	0.7879	**+1.62 pp**
Macro Precision	**0.7951**	0.7825	**+1.26 pp**
Macro Recall	**0.7903**	0.7704	**+1.98 pp**
Macro F1	**0.7917**	0.7718	**+1.99 pp**

CE performs better on every reported aggregate metric.

The largest difference is in Macro-F1:

0.7917 - 0.7718 = 0.0199

or approximately **1.99 percentage points**.

---

11 Confidence Behavior

The two loss functions also produce fundamentally different output scores.

11.1 CE

CE uses softmax to convert logits into a probability distribution:

```
probabilities = softmax(logits)
```

The eight probabilities sum to 1.

For example, a CE prediction can produce:

```
[0.000004, 0.002646, 0.021352, 0.963950,
 0.011806, 0.000016, 0.000093, 0.000133]
```

The maximum value represents the model's relative probability for the selected class.

11.1.1 BCE

BCE uses independent sigmoid outputs:

```
probabilities = sigmoid(logits)
```

These values do not have to sum to 1.

For example:

```
[0.000150, 0.025038, 0.332890, 0.857250,
 0.002555, 0.000480, 0.001040, 0.001731]
```

Therefore, a BCE value such as `0.857` should not be interpreted in exactly the same way as a CE softmax probability of `0.857`.

This difference is important when using confidence thresholds for later `needs_review` or unknown-class detection.

---

12 Prediction Error Analysis

The confidence examples demonstrate that high confidence does not necessarily imply correct classification.

For example:

```
True = minibus
Predicted = kamyun
Confidence = 0.9539
```

and:

```
True = kamyun
Predicted = taxi
Confidence = 0.9885
```

These are highly confident incorrect predictions.

This is especially relevant to the project because confidence-based rejection is intended to identify potentially unreliable predictions. A high softmax confidence alone cannot guarantee that a prediction is correct.

The confusion matrices also show that the most problematic errors occur between visually similar classes, particularly:

```
kamyun $\leftrightarrow$ kamyunet
```

Therefore, a future unknown-class or `needs_review` mechanism should not be evaluated only by overall accuracy.

---

13 Interpretation

The task is a **single-label multiclass classification problem**:

```
one image $\rightarrow$ exactly one vehicle class
```

This structure matches CrossEntropyLoss naturally.

BCE, on the other hand, treats each output independently. That formulation is more natural for multi-label problems where multiple classes may simultaneously be correct.

The experimental results are consistent with this distinction.

CE achieved higher:

- Accuracy
- Macro Precision
- Macro Recall
- Macro F1

and also provides a normalized probability distribution through softmax, making its output more convenient for the confidence-based analysis used later in the project.

---

14 Conclusion

The experiment demonstrates that **CrossEntropyLoss is the better choice for this 8-class single-label vehicle classification problem**.

CE achieved:

- **80.41% validation accuracy**
- **0.7951 Macro Precision**
- **0.7903 Macro Recall**
- **0.7917 Macro F1**

while BCE achieved:

- **78.79% validation accuracy**
- **0.7825 Macro Precision**
- **0.7704 Macro Recall**
- **0.7718 Macro F1**

The confusion matrices show that CE was particularly stronger for `kamyun`, correctly classifying **70/100** samples compared with **52/100** for BCE.

However, BCE performed better on several other classes, including `kamyunet`, `minibus`, `savari`, and `taxi`. Therefore, the advantage of CE is an overall effect rather than a universal improvement for every class.

14.1 Overall, the results support selecting CrossEntropyLoss for the final multi-class vehicle classifier. Its formulation matches the mutually exclusive nature of the classes, provides normalized softmax probabilities, and achieved better overall validation performance in this controlled experiment.

15 Experiment 09 — Global Average Pooling (GAP)

15.1 Objective

In this experiment, we investigate the effect of **Global Average Pooling (GAP)** on the CNN's performance and overfitting.

Instead of flattening the entire feature maps and passing a large number of features to the fully connected layers, GAP computes the average value of each feature map and reduces it to a single value.

The main goal is to determine whether reducing the number of parameters and the capacity of the classifier can reduce overfitting and improve generalization on the validation set.

All experimental conditions were kept fixed, and only the use of GAP was changed.

15.2 Experimental Setup

The CNN architecture contains two convolutional blocks followed by a fully connected classifier.

Without GAP, the feature maps are flattened before being passed to the classifier.

After the convolutional and pooling layers, the feature representation has approximately:

```
64 x 56 x 56
```

Therefore, without GAP, flattening produces:

```
64 x 56 x 56 = 200,704 features
```

These features are then passed to:

```
200,704 $\rightarrow$ 128 $\rightarrow$ 8
```

With GAP, each of the 64 feature maps is reduced to a single average value:

```
64 x 56 x 56
       ↓
Global Average Pooling
       ↓
64 x 1 x 1
       ↓
64 features
       ↓
128 $\rightarrow$ 8
```

Thus, GAP substantially reduces the number of features entering the classifier and consequently reduces the number of trainable parameters in the fully connected layers.

The project specification requires controlled ablations where one factor is changed at a time. GAP was therefore evaluated as a separate controlled experiment.

15.3 Training Results

The model was trained for a maximum of 50 epochs with early stopping.

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Learning Rate
1	1.9828	0.2290	1.8512	0.3314	0.001000
2	1.8144	0.3027	1.7874	0.3417	0.001000
3	1.7304	0.3481	1.6849	0.3652	0.001000
4	1.6584	0.3809	1.5921	0.4212	0.001000
5	1.5973	0.4027	1.5603	0.4227	0.001000
6	1.5738	0.3945	1.5343	0.4345	0.001000
7	1.5293	0.4255	1.4911	0.4462	0.001000
8	1.4979	0.4333	1.4512	0.4757	0.001000
9	1.4533	0.4602	1.4199	0.4698	0.001000
10	1.4244	0.4701	1.3893	0.4831	0.001000
11	1.3780	0.4819	1.3366	0.5022	0.001000
12	1.3562	0.4889	1.3980	0.4742	0.001000
13	1.3348	0.5159	1.2999	0.5346	0.001000

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Learning Rate
...	...	...	...	...	...
22	—	—	—	**0.5847**	0.001000
25	—	—	—	—	0.001000

Early stopping was triggered at **epoch 25**.

The best validation accuracy was:

```
Best validation accuracy = 0.5847
Best epoch = 22
```

15.4 Validation Results

The best GAP checkpoint was evaluated on the validation set.

```
Number of validation predictions: 679
Number of correct validation predictions: 397
Validation accuracy: 0.5846833578792342
```

Therefore:

Metric	GAP
Accuracy	**58.47%**
Macro Precision	**52.81%**
Macro Recall	**53.87%**
Macro F1	**52.69%**

15.5 Classification Report

Class	Precision	Recall	F1-score	Support
ambulance	0.6061	0.5556	0.5797	72
autobus	0.5049	0.5532	0.5279	94
kamyun	0.6944	0.5000	0.5814	100
kamyunet	0.3448	0.5495	0.4237	91
minibus	0.6176	0.5122	0.5600	82
savari	0.6372	0.7200	0.6761	100
taxi	0.8198	0.9192	0.8667	99
vanet	0.0000	0.0000	0.0000	41
Accuracy			**0.5847**	**679**
Macro Avg	**0.5281**	**0.5387**	**0.5269**	**679**
Weighted Avg	**0.5706**	**0.5847**	**0.5705**	**679**

15.5.1 Confusion matrix

True\Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	40	4	0	20	30	8	0	0
autobus	5	52	3	18	7	2	7	0
kamyun	0	15	50	25	3	2	4	1
kamyunet	6	8	8	50	13	3	3	0
minibus	0	11	8	15	42	4	2	0
savari	9	8	0	7	1	72	3	0
taxi	1	3	0	2	1	1	91	0
vanet	5	2	3	8	1	21	1	0

15.5.2 Confusion Matrix Analysis

The confusion matrix shows that the GAP model struggles mainly with visually similar vehicle classes.

The most noticeable confusion occurs between **kamyun and kamyunet**, where a considerable number of samples from both classes are misclassified as the other class. There is also substantial confusion between **ambulance and kamyunet/minibus**, as well as between **vanet and savari**.

The model performs relatively well on **taxi** and **savari**, while **vanet** is the most difficult class. In particular, none of the 41 `vanet` validation samples were correctly classified.

Overall, the confusion matrix supports the previous observation that GAP may have removed useful spatial information and reduced the model's ability to distinguish between visually similar vehicle categories.

15.6 Per-Class Analysis

The results show substantial differences between the classes.

15.6.1 Taxi

The `taxi` class achieved the strongest performance:

```
Precision = 0.8198
Recall    = 0.9192
F1        = 0.8667
```

This indicates that the GAP model was relatively effective at identifying taxis.

The high recall means that most actual taxi images were correctly recognized.

15.6.2 Savari

The `savari` class also performed relatively well:

```
Precision = 0.6372
Recall    = 0.7200
F1        = 0.6761
```

Its recall was the second-highest among the classes.

15.6.3 Kamyun

The `kamyun` class achieved:

```
Precision = 0.6944
Recall    = 0.5000
F1        = 0.5814
```

Although its precision was relatively high, its recall was only 50%.

This means that the model could identify `kamyun` reasonably well when it predicted that class, but it missed many actual `kamyun` samples.

15.6.4 Kamyunet

The `kamyunet` class had particularly weak precision:

```
Precision = 0.3448
Recall    = 0.5495
F1        = 0.4237
```

The low precision indicates that many images predicted as `kamyunet` actually belonged to other classes.

15.6.5 Vanet

The weakest class was `vanet`:

```
Precision = 0.0000
Recall    = 0.0000
F1        = 0.0000
```

The model failed to correctly identify any validation examples of this class.

This is especially important because `vanet` is one of the smaller classes in the validation set, with only 41 examples.

15.7 Effect of GAP on the Feature Representation

The main architectural difference introduced by GAP is the removal of most spatial information before the fully connected classifier.

Without GAP:

```
64 x 56 x 56
```

is converted into:

```
200,704 features
```

The classifier therefore receives information about both:

1. Which features are present.
2. Where those features occur in the feature maps.

With GAP:

```
64 x 56 x 56
        ↓
64 values
```

Only the average activation of each feature map is retained.

Therefore, information about the exact spatial location of features is largely removed.

For vehicle classification, this can be important because visual characteristics may depend not only on which features exist, but also on their spatial arrangement.

For example, the location and arrangement of windows, wheels, body structure, lights, and other visual components may help distinguish visually similar vehicle classes.

Consequently, the strong reduction in representation size may have removed information that was useful for this particular classification task.

15.8 GAP and Overfitting

One motivation for using GAP was to reduce the classifier's capacity and potentially reduce overfitting.

This is a reasonable hypothesis because the fully connected layer without GAP receives approximately 200,704 input features, while the GAP version receives only 64.

However, the experiment shows that **reducing model capacity did not improve validation performance**.

Instead, the best validation accuracy was only:

```
58.47\%
```

This suggests that the model may have become too constrained to represent the classification problem effectively.

In other words, reducing the number of parameters does not automatically lead to better generalization.

There is a trade-off:

```
Too much capacity
        ↓
Potential overfitting

Appropriate capacity
        ↓
Good learning + good generalization
```

```
Too little capacity
        ↓
Underfitting
```

The GAP experiment appears to move the model toward the third case.

15.9 Underfitting vs. Overfitting

The training behavior also suggests that the problem is not simply excessive overfitting.

The model's training accuracy remains relatively low compared with the performance achieved by the stronger CNN configurations used in previous experiments.

The best validation accuracy is only 58.47%, and the model does not reach the very high training accuracy observed in some of the other experiments.

Therefore, the results are more consistent with **insufficient model capacity / underfitting** than with a successful regularization strategy.

This is an important distinction:

> GAP may reduce the model's ability to memorize the training data and prevent the model from overfitting, but in this experiment it also reduced the model's ability to learn the underlying classification task effectively and caused underfitting.

15.10 Comparison with Previous Experiments

The GAP model can be compared with the stronger validation results obtained in previous experiments.

Experiment	Validation Accuracy
GAP	**58.47%**
Balanced Batches	73.64%
Scheduler	78.65%
CrossEntropy	80.41%

The GAP model achieved the lowest validation accuracy among these experiments.

This indicates that, under the current experimental conditions, replacing the flattening operation with GAP substantially reduced classification performance.

However, these results should be interpreted as a conclusion about this specific CNN architecture and dataset rather than a general statement that GAP is ineffective.

15.11 Main Findings

The experiment produced several important observations:

1. **GAP substantially reduces the classifier input size.**

   The representation is reduced from approximately 200,704 features to only 64 features.

2. **The reduction in parameters did not improve validation performance.**

   The best validation accuracy was only 58.47%.

3. **The model appears to have insufficient capacity.**

   The relatively low training and validation performance suggests that the model may be underfitting.

4. **Spatial information appears to be important for this task.**

   GAP removes much of the spatial information contained in the feature maps.

5. **The effect is class-dependent.**

   Taxi performed relatively well, while vanet was not correctly recognized at all.

6. **Kamyun and kamyunet remain difficult classes.**

   Their performance suggests that the visual representation learned by the GAP model is not sufficient to reliably distinguish these classes.

7. **Reducing parameters alone is not enough to solve overfitting.**

   Regularization must preserve enough model capacity to learn meaningful patterns.

15.12 Conclusion

The goal of Experiment 09 was to investigate whether Global Average Pooling could reduce overfitting and improve validation generalization by substantially reducing the capacity of the fully connected classifier.

The results do not support this hypothesis for the current CNN architecture.

The GAP model achieved a best validation accuracy of **58.47%**, with a macro precision of **52.81%**, macro recall of **53.87%**, and macro F1 of **52.69%**.

Compared with the stronger configurations from previous experiments, GAP performed substantially worse.

The main explanation is that GAP reduces the representation from approximately **200,704 features to only 64 features**, removing a large amount of spatial information before classification. Although this reduces the number of parameters and may reduce the risk of overfitting, in this case the reduction appears to be too aggressive.

Therefore, the experiment suggests that the original classifier has useful capacity that should not be removed entirely. The poor performance of GAP is more consistent with **underfitting or insufficient representational capacity** than with successful regularization.

Based on this controlled experiment, **GAP is not selected for the final CNN configuration**.

Importantly, this conclusion applies to the current architecture, dataset, and training setup. It does not imply that GAP is generally inferior to flattening. GAP can be highly effective in other CNN architectures, particularly when the preceding convolutional layers have already learned sufficiently strong high-level representations.

---

16 Experiment 10 — Transfer Learning with ResNet18

16.1 Objective

In this experiment, we investigate whether an ImageNet-pretrained **ResNet18** can improve traffic-vehicle classification compared with the CNN models developed earlier.

Three transfer-learning strategies are evaluated:

1. **Feature Extraction:** the pretrained ResNet18 backbone is frozen and only the new classification head is trained.
2. **Partial Fine-Tuning:** after training the classification head, `layer4` and the classification head are unfrozen and trained using different learning rates.
3. **Full Fine-Tuning:** all ResNet18 layers are unfrozen and trained, using a smaller learning rate for the pretrained backbone and a larger learning rate for the classification head.

An additional experiment investigates the effect of removing ImageNet normalization during full fine-tuning.

The experiments use the same training/validation split and validation-based model selection as the previous experiments. The final frozen test set is not used for model or hyperparameter selection.

16.2 Experimental Setup

The standard ImageNet-pretrained ResNet18 architecture is preserved. The original convolutional stem, including `conv1` and `maxpool`, is kept unchanged.

Because the original ResNet18 classifier predicts 1000 ImageNet classes, its final fully connected layer is replaced with a classifier for the eight vehicle classes.

Images are resized to `224x224`. The main transfer-learning experiments use ImageNet normalization.

16.3 1. ResNet18 Feature Extraction

In the feature-extraction setting, all pretrained ResNet18 parameters are frozen and only the newly initialized classification layer is trained.

This allows the experiment to test whether the visual representation learned from ImageNet is already sufficiently useful for distinguishing the eight traffic-vehicle classes.The model contains approximately **11.18 million total parameters**, but only **4,104 parameters** are trainable:

$$512 \times 8 + 8 = 4104$$

The model was trained with a learning rate of `0.001`.

16.3.1 Training Results

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	1.4689	0.5195	0.9746	0.7216
2	0.8531	0.7607	0.7262	0.7968
3	0.6806	0.8119	0.6703	0.7997
4	0.5829	0.8355	0.5608	0.8321
5	0.5300	0.8466	0.5301	0.8292
6	0.4838	0.8628	0.4915	0.8527
7	0.4584	0.8617	0.4861	0.8542
8	0.4375	0.8632	0.4563	0.8527
9	0.4023	0.8702	0.4428	0.8557
10	0.3939	0.8809	0.4660	0.8365
11	0.3799	0.8868	0.4203	**0.8689**
12	0.3439	0.9041	0.4264	0.8601
13	0.3393	0.8956	0.4214	0.8645
14	0.3387	0.8964	0.4170	0.8689
15	0.3163	0.9001	0.4136	0.8689
16	0.3104	0.9104	0.4235	0.8630

Early stopping was triggered at epoch 16.

Best validation accuracy: 86.89% at epoch 11

16.3.2 Classification Performance

The feature-extraction model achieved:

- Accuracy: **86.89%**
- Macro Precision: **86.18%**
- Macro Recall: **85.87%**
- Macro F1: **85.91%**
- Weighted F1: **86.79%**

The pretrained representation is therefore already effective for this classification task, even though only 4,104 parameters are trained.

16.4 2. Partial Fine-Tuning — Layer4 + Classification Head

Feature extraction keeps the entire pretrained representation fixed. To allow the model to adapt more specifically to vehicle images, `layer4` is subsequently unfrozen together with the classification head.

The earlier ResNet blocks remain frozen, while the deeper `layer4` features are allowed to adapt.

A smaller learning rate is used for the pretrained `layer4` parameters, while the newly initialized classification head uses a larger learning rate.

16.4.1 Training Results

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.3127	0.8956	0.2901	0.8999
2	0.0419	0.9919	0.2624	0.9249
3	0.0110	0.9993	0.2502	0.9278
4	0.0053	1.0000	0.2473	0.9308
5	0.0030	1.0000	0.2439	0.9293
6	0.0028	1.0000	0.2528	0.9337
7	0.0017	1.0000	0.2568	0.9323
8	0.0011	1.0000	0.2602	0.9352
9	0.0009	1.0000	0.2551	0.9352
10	0.0006	1.0000	0.2667	0.9367
11	0.0007	1.0000	0.2679	0.9337
12	0.0007	1.0000	0.2703	**0.9396**
13	0.0187	0.9956	0.4681	0.9057
14	0.0238	0.9934	0.3295	0.9278
15	0.0127	0.9956	0.4048	0.9367
16	0.0155	0.9952	0.3582	0.9234
17	0.0297	0.9912	0.7378	0.9013

Early stopping was triggered at epoch 17.

Best validation accuracy: 93.96% at epoch 12

The improvement from 86.89% to 93.96% demonstrates that allowing the deeper pretrained features to adapt substantially improves performance.

16.5 3. Full Fine-Tuning

In the full fine-tuning experiment, all ResNet18 parameters are trainable.

The pretrained backbone uses a smaller learning rate, while the classification head uses a larger learning rate. This allows the pretrained representation to adapt gradually without changing it too aggressively.

The validation results were:

Strategy	Validation Accuracy	Macro F1	Trainable Parameters
Feature Extraction	86.89%	85.91%	4,104
Layer4 + FC Fine-Tuning	93.96%	93.77%	8,397,832
Full Fine-Tuning	**96.02%**	**95.73%**	11,180,616

Full fine-tuning produced the best result among the three transfer-learning strategies.

16.5.1 Full Fine-Tuning Classification Results

Class	Precision	Recall	F1
ambulance	1.00	0.99	0.99
autobus	0.99	0.99	0.99
kamyun	0.94	0.92	0.93

Class	Precision	Recall	F1
kamyunet	0.90	0.97	0.93
minibus	0.96	0.96	0.96
savari	0.97	0.96	0.96
taxi	1.00	0.97	0.98
vanet	0.90	0.90	0.90

Overall:

- Accuracy: **96.02%**
- Macro Precision: **95.77%**
- Macro Recall: **95.73%**
- Macro F1: **95.73%**
- Weighted F1: **96.04%**

16.6 4. Effect of ImageNet Normalization

An additional full fine-tuning experiment was performed without ImageNet normalization.

The best validation accuracy was:

96.32% at epoch 8

This is slightly higher than the **96.02%** obtained with the standard normalization.

Full Fine-Tuning Setup	Best Val Accuracy
With ImageNet normalization	96.02%
Without normalization	**96.32%**

The difference is only **0.30 percentage points**, so the result should not be interpreted as strong evidence that removing normalization is generally better.

Rather, it shows that for this particular dataset and training configuration, the model achieved slightly better validation performance without normalization.

Since the model uses ImageNet-pretrained weights, ImageNet normalization remains the standard and conceptually consistent preprocessing choice for the main transfer-learning experiment.

16.7 Analysis

The results show a clear progression as more of the pretrained network is allowed to adapt to the target dataset.

Feature extraction achieved **86.89%** validation accuracy while training only 4,104 parameters. This demonstrates that the pretrained ImageNet representation already contains useful visual information for vehicle classification.

Allowing `layer4` to adapt increased validation accuracy to **93.96%**. This indicates that the higher-level features learned by the pretrained network can benefit from task-specific adaptation.

Full fine-tuning produced the best performance at **96.02%**, showing that adapting the entire network provided additional improvement over partial fine-tuning.

The improvement is substantial:

- Feature Extraction → Layer4 Fine-Tuning: **+7.07 percentage points**
- Layer4 Fine-Tuning → Full Fine-Tuning: **+2.06 percentage points**
- Feature Extraction → Full Fine-Tuning: **+9.13 percentage points**

The training curves also show that fine-tuning has substantially more capacity. In the layer4 fine-tuning experiment, training accuracy reached approximately 100% very quickly while validation accuracy continued improving until epoch 12. After that point, validation performance became less stable and eventually deteriorated, which explains the use of early stopping.

16.8 Comparison with Previous CNN Baseline

The transfer-learning experiments substantially outperform the **plain CNN baseline** developed in Experiment 01.

The CNN baseline achieved a best validation accuracy of **78.94%**, whereas ResNet18 feature extraction already achieved **86.89%** while training only 4,104 parameters.

Allowing the pretrained ResNet18 to adapt to the target dataset further improved performance. Partial fine-tuning of `layer4` increased validation accuracy to **93.96%**, while full fine-tuning achieved **96.02%**.

The comparison can be summarized as follows:

Model	Validation Accuracy	Macro F1
CNN Baseline	78.94%	77.56%
ResNet18 Feature Extraction	86.89%	85.91%
ResNet18 Layer4 Fine-Tuning	93.96%	93.77%
ResNet18 Full Fine-Tuning	**96.02%**	**95.73%**
ResNet18 Full Fine-Tuning without normalization	**96.32%**	—

Compared with the CNN baseline, ResNet18 feature extraction improves validation accuracy by **7.95 percentage points**, even though only the final classification layer is trained.

Partial fine-tuning provides an additional improvement of **7.07 percentage points** over feature extraction, while full fine-tuning improves performance by another **2.06 percentage points**.

Overall, full fine-tuning improves validation accuracy by approximately **17.08 percentage points** compared with the CNN baseline.

These results demonstrate the benefit of using a strong pretrained visual representation. Instead of learning all visual features from scratch, ResNet18 starts with features learned from ImageNet and then adapts them to the traffic-vehicle classification task.

The comparison also shows that the improvement is not only due to replacing the classifier. Feature extraction already improves substantially over the CNN baseline, and progressively unfreezing more of the pretrained network leads to further gains.

16.9 Main Findings

1. **Pretrained ResNet18 provides a major advantage** over the CNN models trained from scratch.
2. **Feature extraction is already strong**, reaching 86.89% while training only 4,104 parameters.
3. **Partial fine-tuning improves performance substantially**, increasing validation accuracy to 93.96%.
4. **Full fine-tuning achieves the best standard transfer-learning result**, reaching 96.02%.
5. Allowing the entire pretrained network to adapt is beneficial for this dataset, although it also increases the risk of overfitting.
6. Removing ImageNet normalization produced a slightly higher validation accuracy of 96.32%, but the improvement is small and should be treated as a dataset-specific experimental result rather than a general conclusion.
7. Early stopping is important because training accuracy reaches nearly 100% while validation performance eventually stops improving.

16.10 Conclusion

The experiment demonstrates that transfer learning with ImageNet-pretrained ResNet18 is highly effective for the traffic-vehicle classification task.

Feature extraction provides a strong baseline with very few trainable parameters, but adapting the pretrained representation to the target dataset produces significantly better results. Partial fine-tuning of `layer4` improves performance further, while full fine-tuning achieves the highest validation accuracy among the standard transfer-learning configurations.

The best standard configuration achieved **96.02% validation accuracy and 95.73% macro F1**, making full fine-tuning the strongest candidate for the final model.

16.11 The additional experiment without ImageNet normalization reached 96.32%, only 0.30 percentage points higher. Therefore, this small difference alone is not sufficient to establish that removing normalization is generally preferable.

16.12 Experiment 11 — ResNet18 with Different Seeds and Different Augmentations

16.12.1 Objective

The objective of this experiment is to investigate how sensitive the ResNet18 model is to the choice of random seed and data augmentation strategy.

Since training a deep learning model involves several sources of randomness, including parameter initialization, data shuffling, and stochastic data augmentation, a single training run may not fully represent the model's performance. Therefore, five different random seeds (`7`, `10`, `30`, `42`, and `86`) were evaluated with six augmentation strategies.

The goal was to determine whether some augmentation strategies consistently improve validation performance and whether the observed performance is stable across different random seeds.

16.12.2 Experimental Setup

The experiments used the pretrained ResNet18 model with full fine-tuning. Images were resized to `224 × 224` and normalized using the standard ImageNet mean and standard deviation.

Five random seeds were evaluated:

- 7
- 10
- 30
- 42
- 86

Six augmentation strategies were compared:

1. **Basic** — Random Horizontal Flip + Random Rotation
2. **Color** — ColorJitter
3. **Crop + Flip** — RandomResizedCrop + Random Horizontal Flip
4. **Rotation + Noise** — Random Rotation + Gaussian Noise
5. **Blur + Erasing** — Gaussian Blur + Random Erasing
6. **Full** — combination of several augmentation techniques

Early stopping was used during training, and the best validation accuracy for each experiment was recorded.

16.12.3 Results

The best validation accuracy obtained for each seed and augmentation strategy is shown below.

Augmentation	Seed 7	Seed 10	Seed 30	Seed 42	Seed 86
Basic	98.23%	97.50%	97.35%	97.94%	97.20%
Color	97.50%	96.76%	96.91%	97.05%	96.47%
Crop + Flip	97.79%	97.79%	97.94%	97.64%	97.64%
Rotation + Noise	95.88%	96.47%	97.05%	96.76%	97.05%
Blur + Erasing	97.05%	97.35%	97.50%	97.35%	96.61%
Full	97.35%	97.64%	97.35%	97.50%	97.64%

16.12.4 Average Performance Across Seeds

To compare the augmentation strategies more reliably, the mean validation accuracy across the five seeds was calculated.

Augmentation	Mean Validation Accuracy	Best Result	Worst Result
Basic	**97.64%**	**98.23%**	97.20%
Crop + Flip	97.76%	97.94%	**97.64%**
Full	97.50%	97.64%	97.35%
Blur + Erasing	97.17%	97.50%	96.61%
Color	96.94%	97.50%	96.47%
Rotation + Noise	96.64%	97.05%	95.88%

16.12.5 Analysis

The results show that the choice of augmentation has a noticeable effect on validation performance, while the effect of the random seed is comparatively smaller.

The **Crop + Flip** strategy achieved the highest average validation accuracy at **97.76%**. More importantly, its results were highly consistent across all five seeds, ranging only from **97.64% to 97.94%**. This indicates that this augmentation strategy provides both strong performance and good stability.

The **Basic** augmentation strategy achieved the highest individual result of **98.23%** with seed 7 and also obtained a strong average accuracy of **97.64%**. However, its performance varied slightly more across seeds than Crop + Flip.

The **Full** augmentation strategy did not outperform the simpler augmentation methods. Despite combining six augmentation operations, its average accuracy was **97.50%**. This suggests that adding more augmentation techniques does not necessarily lead to better performance. Some transformations may introduce unnecessary variation that does not match the real distribution of traffic-vehicle images.

The **Color** augmentation strategy produced a lower average accuracy of **96.94%**. This may indicate that color and contrast variations alone were not as beneficial as geometric transformations for this dataset.

The **Rotation + Noise** strategy achieved the lowest average accuracy at **96.64%** and also had the lowest individual result, **95.88%**. Adding artificial noise appears to have been less suitable for this dataset compared with the other augmentation strategies.

Overall, the results suggest that **moderate and task-relevant augmentation is more effective than simply increasing the number of augmentation operations**. Geometric transformations such as cropping, flipping, and limited rotation appear to provide useful variation without significantly changing the semantic content of the vehicles.

16.12.6 Effect of Random Seed

The results also demonstrate that random seed can affect the final validation accuracy, but the magnitude of this effect depends on the augmentation strategy.

For example, the Basic augmentation ranged from **97.20% to 98.23%**, while Crop + Flip varied only from **97.64% to 97.94%**.

Therefore, evaluating multiple seeds provides a more reliable estimate of model performance than reporting the result of a single run. In particular, the small variation observed for Crop + Flip suggests that its performance is relatively robust to the random seed.

16.12.7 Best Augmentation Strategy

Based on the mean validation accuracy across the five seeds, **Crop + Flip** was the strongest overall augmentation strategy:

- Mean validation accuracy: **97.76%**
- Best validation accuracy: **97.94%**
- Worst validation accuracy: **97.64%**
- Range across seeds: **0.30 percentage points**

Although Basic augmentation achieved the highest single-run accuracy (**98.23%**), Crop + Flip

produced more consistent results across different seeds.

Therefore, if the objective is to select an augmentation strategy based on both **performance and stability**, Crop + Flip is the strongest candidate from this experiment.

16.12.8 Conclusion

Experiment 11 demonstrates that both augmentation strategy and random seed can influence the performance of a fine-tuned ResNet18 model.

However, the results indicate that **the type of augmentation is more important than the particular random seed**. The best results were obtained with relatively simple geometric transformations rather than with increasingly complex combinations of augmentation techniques.

Among the tested strategies, **Crop + Flip achieved the highest average validation accuracy of 97.76% and showed the smallest variation across seeds**. Basic augmentation achieved the highest individual result of 98.23%, but was slightly less stable.

These findings support using **Crop + Flip** as a strong and robust augmentation strategy for the final model, subject to confirmation on the held-out evaluation set only after all model and augmentation cho

---

17 Experiment 12 — Letterbox Preprocessing on the Training Dataset With Dropout

17.1 Objective

The objective of this experiment was to investigate the effect of **letterbox preprocessing on the training dataset** when using a fully fine-tuned ImageNet-pretrained ResNet18.

Two preprocessing strategies were compared:

1. **Letterbox preprocessing**, which preserves the original aspect ratio of the vehicle images.
2. **Non-letterbox preprocessing**, in which images are directly resized to 224x224.

Both configurations used the same model architecture, augmentation, dropout, optimizer, weight decay, learning-rate scheduler, and random seed. The main difference between the two experiments was therefore the preprocessing method applied to the training images.

The experiment also evaluated the model using **Grad-CAM error analysis** to investigate the visual regions associated with misclassified validation samples.

17.2 Experimental Setup

An **ImageNet-pretrained ResNet18** was used for the experiment. All layers of the network were unfrozen, so the model was trained using **full fine-tuning**.

The final fully connected layer was replaced with a classifier containing a dropout layer followed by an 8-class linear layer.

17.2.1 Model and Training Configuration

Parameter	Setting
Architecture	ResNet18
Pretrained weights	ImageNet
Training strategy	Full fine-tuning
Number of classes	8
Input size	224 x 224
Trainable layers	All layers
Dropout	0.3
Optimizer	Adam
Backbone learning rate	1e-4
Head learning rate	1e-4
Weight decay	1e-4
Scheduler	ReduceLROnPlateau
Scheduler mode	`max`
Scheduler factor	0.03
Scheduler patience	10
Batch size	32
Maximum epochs	500
Random seed	7
Normalization	ImageNet

The experiments were executed using a CUDA-enabled GPU.

17.3 Data Preprocessing

Two different training transformations were evaluated.

17.3.1 Letterbox Training

In the first configuration, each image was processed using letterbox preprocessing.

The image was proportionally resized to fit inside a **224x224** frame. The remaining regions were padded so that the original aspect ratio was preserved.

After letterbox preprocessing, the following augmentations were applied:

- Random horizontal flip with probability 0.5
- Random rotation of ± 10 degrees
- Conversion to tensor
- ImageNet normalization

The main advantage of this approach is that the vehicle is not geometrically stretched when converting images with different aspect ratios into a square input.

17.3.2 Non-Letterbox Training

In the second configuration, the images were directly resized to:

```
224 x 224
```

without explicitly preserving their original aspect ratio.

The same augmentations were then applied:

- Random horizontal flip
- Random rotation of ± 10 degrees
- ImageNet normalization

Therefore, the two configurations were controlled so that the primary difference was the presence or absence of **letterbox preprocessing during training**.

---

18 Model Architecture

The model used ImageNet-pretrained ResNet18 and replaced the original classification head with:

```
ResNet18 backbone
       ↓
Dropout(p = 0.3)
       ↓
Linear layer
       ↓
8 output classes
```

All parameters were trainable.

Using full fine-tuning allowed both the pretrained feature extractor and the final classification layer to adapt to the traffic-vehicle dataset.

The dropout layer was applied only to the classification head and was active during training. During evaluation, dropout was automatically disabled.

---

19 Training Procedure

Both models were initialized independently from the same ImageNet-pretrained ResNet18 weights using **seed 7**.

The models were trained using Adam with a learning rate of `1e-4` for both the backbone and classification head. Weight decay of `1e-4` was also applied.

`ReduceLROnPlateau` was used to adapt the learning rate when validation performance stopped improving. Early stopping was configured with a patience of 75 epochs.

The best model was selected according to **validation accuracy**, rather than the final epoch.

This is important because the model continued training after reaching its best validation accuracy, but later epochs did not necessarily provide better generalization.

---

20 Results

The two preprocessing strategies produced the following validation results:

Model	Preprocessing	Best Validation Accuracy
ResNet18 Full Fine-tuning	Letterbox	**97.79%**
ResNet18 Full Fine-tuning	Non-Letterbox	**98.38%**

The non-letterbox configuration achieved the higher validation accuracy.

The difference was:

$$98.38\% - 97.79\% = 0.59\%$$

Therefore, direct resizing outperformed letterbox preprocessing by approximately **0.59 percentage points** in this experiment.

20.1 Letterbox Results

The letterbox configuration achieved a best validation accuracy of:

97.79%

at **epoch 37**.

The training accuracy became extremely high during training, eventually reaching approximately 100%. At the same time, validation accuracy remained around the upper-97% range.

This indicates that the model had enough capacity to fit the training dataset very effectively, while its validation performance had already reached its best level relatively early in training.

The later training epochs did not produce a new best validation accuracy, and early stopping was eventually triggered at epoch 112.

Therefore, the best checkpoint rather than the final training state should be used for evaluation.

20.2 Non-Letterbox Results

The non-letterbox model achieved a best validation accuracy of:

98.38%

at **epoch 70**.

The model continued training beyond this point, but subsequent epochs did not improve the best validation accuracy. Early stopping was eventually triggered at epoch 145.

The best non-letterbox model therefore achieved a slightly better validation result than the letterbox model.

---

21 Direct Comparison

The direct comparison is:

Preprocessing	Best Val Accuracy
Letterbox	97.79%
Non-Letterbox	**98.38%**

The non-letterbox configuration achieved an improvement of approximately **0.59 percentage points**.

This result suggests that preserving the original aspect ratio through letterbox preprocessing **did not provide an accuracy advantage for this dataset under the current configuration**.

However, the difference is still relatively small. Therefore, this experiment alone is not sufficient to conclude that non-letterbox preprocessing is universally superior.

---

22 Per-Class Performance

The reported validation classification results for the selected non-letterbox model were:

Class	Precision	Recall	F1-score
ambulance	1.00	1.00	1.00
autobus	1.00	1.00	1.00
kamyun	0.99	0.98	0.98
kamyunet	0.96	0.99	0.97
minibus	0.98	0.98	0.98
savari	0.97	1.00	0.99
taxi	0.99	0.98	0.98
vanet	1.00	0.90	0.95

Overall:

- **Accuracy:** 98.38%
- **Macro precision:** 98.55%
- **Macro recall:** 97.84%
- **Macro F1:** 98.15%
- **Weighted F1:** 98.37%

The results demonstrate strong performance across all eight classes.

The lowest recall belonged to `vanet` at approximately **90%**, indicating that some `vanet` samples were still classified as other vehicle classes. Nevertheless, its precision was 100%, meaning that predictions assigned to `vanet` were highly reliable in this validation result.

---

23 Effect of Dropout

Unlike the previous experiment without dropout, this configuration used:

```
Dropout = 0.3
```

in the ResNet18 classification head.

Dropout introduces stochastic removal of a proportion of classifier activations during training, which acts as a regularization mechanism.

Despite using dropout, the model still reached very high validation accuracy. Therefore, dropout did not prevent the ResNet18 model from learning highly discriminative features.

The comparison with the corresponding no-dropout experiment should be interpreted carefully because the experiments may differ in other configuration details. However, the current results demonstrate that **dropout = 0.3 is compatible with strong performance in the full fine-tuning setup**.

---

24 Training Behavior and Overfitting

The training accuracy eventually approached 100%, while validation accuracy remained below training accuracy.

This indicates that the model was able to almost completely fit the training data.

However, the validation accuracy did not continue increasing indefinitely. Once the model reached its best validation performance, additional training produced fluctuations rather than consistent improvement.

This behavior is consistent with a model that has high capacity relative to the dataset and therefore can fit the training samples extremely well.

The use of dropout, weight decay, data augmentation, learning-rate scheduling, and early stopping provides several mechanisms for controlling this behavior.

The important point is that **the best validation checkpoint should be used rather than the final epoch**.

---

25 Grad-CAM Error Analysis

Grad-CAM was used to analyze misclassified validation images.

The purpose of this analysis was to examine **which spatial regions of the input image contributed most strongly to the model's prediction**.

For correctly classified samples, Grad-CAM can indicate whether the model is focusing on meaningful vehicle regions. For incorrectly classified samples, it can help investigate whether the model relied on:

- vehicle shape and structure,

- visually similar parts between vehicle classes,
- background regions,
- contextual information,
- or other potentially irrelevant image regions.

This qualitative analysis is particularly useful because validation accuracy alone does not explain **why** a prediction was incorrect.

The Grad-CAM results should therefore be considered complementary to the quantitative classification metrics.

Since the provided output does not include the actual Grad-CAM visualizations or observations from those visualizations, specific claims about the exact regions attended to by the model cannot be made from the numerical results alone.

---

26 Letterbox vs. Non-Letterbox Analysis

The central question of the experiment was whether preserving aspect ratio during training would improve classification performance.

The results were:

- Letterbox: **97.79%**
- Non-letterbox: **98.38%**

Thus, letterbox preprocessing resulted in slightly lower validation accuracy.

One possible explanation is that the direct resizing distortion introduced by mapping images to 224x224 was not sufficiently harmful to classification. ResNet18 may still extract strong semantic features from the resized images.

Another possibility is that the dataset itself contains sufficient visual information for the network to recognize vehicle categories even when aspect ratios are slightly modified.

Letterbox preprocessing also introduces padded regions into the image. These regions do not contain additional vehicle information and may therefore provide less useful input to the model than a fully occupied 224x224 image.

However, these are possible explanations rather than conclusions directly established by the experiment.

The strongest conclusion supported by the experiment is simply that **letterbox preprocessing did not improve validation accuracy in this particular configuration**.

---

27 Comparison with the Previous No-Dropout Experiment

The previous experiment investigated letterbox and non-letterbox preprocessing **without dropout**.

The current experiment adds dropout with:

```
Dropout = 0.3
```

while keeping the main ResNet18 full fine-tuning framework.

The current comparison remains:

Configuration	Letterbox	Non-Letterbox
With dropout = 0.3	97.79%	**98.38%**

Therefore, even with dropout, the same general pattern was observed: **non-letterbox preprocessing achieved the higher validation accuracy**.

This provides additional evidence that letterbox preprocessing was not particularly beneficial for this dataset under the tested training configuration.

---

28 Main Findings

The main findings of this experiment are:

1. **Full fine-tuning of ImageNet-pretrained ResNet18 achieved excellent validation performance**, with both preprocessing strategies exceeding 97% accuracy.
2. **Non-letterbox preprocessing performed better than letterbox preprocessing**, achieving 98.38% versus 97.79%.
3. The difference was approximately **0.59 percentage points**, which is noticeable but still relatively small.
4. **Dropout with (p=0.3)** was compatible with very strong model performance.
5. Training accuracy approached 100%, showing that the model had enough capacity to fit the training data extremely well.
6. Validation performance did not continue improving throughout training, demonstrating the importance of **early stopping and selecting the best validation checkpoint**.
7. **Grad-CAM was used as a qualitative error-analysis method** to investigate the regions contributing to misclassified predictions.
8. Based on validation accuracy, **non-letterbox preprocessing was the better choice among the two configurations tested in this experiment**.

---

29 Conclusion

This experiment investigated whether **letterbox preprocessing applied during training** could improve the performance of a fully fine-tuned ImageNet-pretrained ResNet18 with dropout.

The results showed that the **non-letterbox model achieved the best validation accuracy of 98.38%**, while the letterbox model achieved **97.79%**. Therefore, direct resizing performed approximately **0.59 percentage points better**.

Although letterbox preprocessing has the advantage of preserving the original aspect ratio and avoiding geometric distortion, this advantage did not translate into higher validation accuracy for the current dataset and model configuration.

The experiment also demonstrated that a ResNet18 model using **dropout = 0.3, data augmentation, weight decay, and ReduceLROnPlateau** can achieve very strong classification performance while using full fine-tuning.

Consequently, based on the validation results, **the non-letterbox preprocessing configuration was preferred for the final model**. However, because the difference between the two approaches is relatively small, this result should be interpreted as a finding specific to the current dataset and experimental setup rather than a general statement about letterbox preprocessing.

---

30 Experiment 13 — Letterbox Preprocessing on the Training Dataset Without Dropout

30.1 Objective

The objective of this experiment was to investigate whether **letterbox preprocessing applied to the training images** improves the performance of a fully fine-tuned ImageNet-pretrained ResNet18.

Two preprocessing strategies were compared:

1. **Letterbox preprocessing**: images were resized while preserving their original aspect ratio, and the remaining area was padded to obtain a 224x224 image.
2. **Non-letterbox preprocessing**: images were directly resized to 224x224.

Both approaches used the same ResNet18 architecture, augmentation strategy, optimizer, learning rates, weight decay, and random seed. The main difference was therefore the preprocessing method used during training.

The experiment was also performed **without dropout** in order to evaluate the effect of letterbox preprocessing without additional regularization from dropout.

30.2 Experimental Setup

A pretrained **ResNet18 with ImageNet weights** was used. All layers of the network were trainable, making this a **full fine-tuning** experiment.

30.2.1 Model Configuration

Parameter	Setting
Architecture	ResNet18
Pretrained weights	ImageNet
Training strategy	Full fine-tuning

Parameter	Setting
Number of classes	8
Input size	224 x 224
Trainable layers	All layers
Dropout	None
Optimizer	Adam
Backbone learning rate	1e-4
Head learning rate	1e-4
Weight decay	1e-4
Scheduler	ReduceLROnPlateau
Scheduler factor	0.03
Seed	7
Batch size	32
Maximum epochs	150
Normalization	ImageNet mean and standard deviation

The model was trained using GPU acceleration with CUDA.

30.3 Data Preprocessing

Two training transformations were compared.

30.3.1 Letterbox preprocessing

For the letterbox experiment, each image was first resized proportionally to fit within a **224x224** frame. The remaining area was filled with padding instead of stretching the image.

The resulting image was then subjected to:

- Random horizontal flip with probability 0.5
- Random rotation of ±10 degrees
- Conversion to tensor
- ImageNet normalization

The important characteristic of this preprocessing is that the **original aspect ratio of the vehicle is preserved**.

30.3.2 Non-letterbox preprocessing

For the second experiment, images were directly resized to:

```
224 x 224
```

without preserving their original aspect ratio.

The same augmentation operations were then applied:

- Random horizontal flip

- Random rotation of ±10 degrees
- ImageNet normalization

Therefore, the main experimental difference was whether the image was **aspect-ratio preserving or directly stretched**.

30.4 Training Procedure

A new ImageNet-pretrained ResNet18 was initialized for each experiment. The final fully connected layer was replaced with an 8-class classifier, and all model parameters were unfrozen.

The models were optimized using Adam with a learning rate of `1e-4` for both the backbone and classification head, together with a weight decay of `1e-4`.

`ReduceLROnPlateau` was used to reduce the learning rate when validation performance stopped improving. Early stopping was also enabled to prevent unnecessary training after a prolonged period without validation improvement.

The model selection criterion was **validation accuracy**, and the best checkpoint was saved according to the highest validation accuracy.

---

31 Results

The comparison between the two preprocessing strategies is shown below.

Model	Preprocessing	Best Validation Accuracy
ResNet18 Full Fine-tuning	Letterbox	**98.09%**
ResNet18 Full Fine-tuning	Non-Letterbox	**98.38%**

The **non-letterbox model achieved the best validation accuracy of 98.38%**, while the letterbox model achieved **98.09%**.

The difference is:

$$98.38\% - 98.09\% = 0.29\%$$

Therefore, the non-letterbox preprocessing produced a small improvement of approximately **0.29 percentage points**.

31.1 Letterbox Results

The letterbox model reached a best validation accuracy of:

98.09%

at **epoch 20**.

The training accuracy eventually reached approximately 100%, while the validation accuracy remained around the high-97% range during the later stages of training.

This indicates that the model was able to fit the training data extremely well. However, the gap between training and validation performance suggests that some degree of overfitting remained.

Importantly, the very high validation accuracy indicates that letterbox preprocessing did not prevent ResNet18 from learning highly discriminative visual features.

31.2 Non-Letterbox Results

The non-letterbox model achieved a higher best validation accuracy:

98.38%

at **epoch 122**.

The model also reached almost perfect training accuracy during training.

Despite the much longer training process, the important result is that the non-letterbox model achieved slightly better validation performance than the letterbox model.

The improvement, however, was relatively small.

---

32 Comparison of Letterbox and Non-Letterbox

The direct comparison is:

Metric	Letterbox	Non-Letterbox
Best validation accuracy	98.09%	**98.38%**
Difference	—	**+0.29 percentage points**

The results do **not** provide evidence that letterbox preprocessing improves classification accuracy for this dataset.

In fact, direct resizing produced the slightly better validation result.

However, the difference is only **0.29 percentage points**, which is relatively small. Therefore, it would not be appropriate to conclude that direct resizing is generally superior to letterbox preprocessing based on this single experiment.

The result instead suggests that, for this particular dataset and model configuration, **preserving the original aspect ratio did not provide an additional accuracy advantage**.

---

33 Analysis

One possible motivation for letterbox preprocessing is that directly resizing an image from a non-square aspect ratio to 224x224 can geometrically distort the vehicle.

For example, a wide vehicle may become relatively taller after direct resizing. Letterbox preprocessing avoids this distortion by maintaining the original aspect ratio.

Despite this theoretical advantage, the validation results show that the model performed slightly better without letterbox preprocessing.

One possible explanation is that ResNet18 is sufficiently powerful to learn useful features despite the geometric distortion introduced by direct resizing. Furthermore, the dataset may contain enough visual variation that the small distortion caused by resizing does not significantly affect classification.

Another important observation is that the difference between the two approaches is very small. Therefore, the experiment should be interpreted as:

> **Letterbox preprocessing did not improve validation accuracy in this experiment.**

rather than:

> **Letterbox preprocessing is worse than direct resizing in general.**

A stronger conclusion would require repeating the comparison with multiple random seeds and examining whether the observed 0.29-point difference remains consistent.

---

34 Training Behavior

Both models achieved extremely high training accuracy, approaching 100%. This demonstrates the high capacity of the fully fine-tuned ResNet18 relative to the size and complexity of the training dataset.

At the same time, the validation accuracy remained slightly below training accuracy. This indicates that the models were capable of fitting the training data more closely than the validation data.

Since dropout was removed in this experiment, the results also show that **very strong validation performance can still be achieved without dropout** when using a pretrained ResNet18 with appropriate augmentation, weight decay, and fine-tuning.

However, the near-perfect training accuracy should not be interpreted as evidence that the model generalizes perfectly. Validation performance is more important for assessing generalization during model selection.

---

35 Per-Class Validation Performance

For the selected non-letterbox model, the validation results were:

Class	Precision	Recall	F1-score
ambulance	1.00	0.99	0.99
autobus	0.99	0.99	0.99
kamyun	0.98	0.99	0.99

Class	Precision	Recall	F1-score
kamyunet	0.98	0.98	0.98
minibus	0.99	0.98	0.98
savari	0.98	1.00	0.99
taxi	0.98	0.99	0.98
vanet	0.97	0.93	0.95

Overall:

- **Accuracy:** 98.38%
- **Macro precision:** 98.37%
- **Macro recall:** 97.95%
- **Macro F1:** 98.15%
- **Weighted F1:** 98.37%

The results show that performance was strong across all eight classes. The `vanet` class had the lowest recall and F1-score, indicating that it remained comparatively more difficult for the model than the other classes.

Nevertheless, its F1-score of **0.95** is still substantially better than the corresponding performance of the earlier CNN baseline.

---

36 Comparison with Previous ResNet18 Experiments

This experiment also provides a useful comparison with the previous full fine-tuning experiments.

The model achieved validation accuracy above 98% with the combination of:

- ImageNet-pretrained ResNet18,
- full fine-tuning,
- data augmentation,
- weight decay,
- learning-rate scheduling,
- and no dropout.

The comparison between the two preprocessing strategies shows that **changing the image preprocessing from direct resizing to letterbox did not improve the final validation accuracy**.

The non-letterbox configuration achieved the highest validation accuracy:

98.38%

while the letterbox configuration achieved:

98.09%

Thus, among these two preprocessing choices, the non-letterbox configuration was selected based on validation accuracy.

---

37 Main Findings

The main findings of this experiment are:

1. **Full fine-tuning of ResNet18 achieved very high validation performance**, exceeding 98% accuracy with both preprocessing strategies.
2. **Letterbox preprocessing did not improve validation accuracy.** The letterbox model achieved 98.09%, compared with 98.38% for direct resizing.
3. **The difference was small**, only approximately 0.29 percentage points, so the experiment does not provide strong evidence that one preprocessing method is universally better.
4. **Removing dropout did not prevent strong generalization** in this ResNet18 configuration. The model still achieved above 98% validation accuracy.
5. **Training accuracy approached 100%**, indicating that the model had sufficient capacity to fit the training data. The remaining difference between training and validation performance indicates some degree of overfitting.
6. **The non-letterbox model produced the best validation result** among the two preprocessing configurations.
7. **Grad-CAM provided an additional qualitative tool for analyzing misclassified samples**, helping investigate what visual regions contributed to the model's decisions.

---

38 Conclusion

This experiment evaluated the effect of **letterbox preprocessing on a fully fine-tuned ImageNet-pretrained ResNet18 without dropout**.

Although letterbox preprocessing preserves the original aspect ratio and theoretically reduces geometric distortion, it did not lead to better validation performance on this dataset. The letterbox model achieved **98.09% validation accuracy**, while direct resizing achieved **98.38%**, giving the non-letterbox configuration a small advantage of **0.29 percentage points**.

Therefore, based on the validation results, **direct resizing was selected over letterbox preprocessing** for the final configuration. However, because the performance difference is small, the result should be considered dataset- and configuration-specific rather than evidence that letterbox preprocessing is generally inferior.

The experiment also demonstrated that a fully fine-tuned pretrained ResNet18 can achieve excellent performance without dropout when combined with augmentation, weight decay, and learning-rate scheduling.

38.1 Dropout vs. No Dropout — Comparison

Model	Preprocessing	Dropout	Best Val Accuracy	Test Accuracy
ResNet18 Full Fine-tuning	Letterbox	0.3	97.79%	—
ResNet18 Full Fine-tuning	Letterbox	0.0	98.09%	—
ResNet18 Full Fine-tuning	Non-Letterbox	0.3	**98.38%**	**97.5%**
ResNet18 Full Fine-tuning	Non-Letterbox	0.0	**98.38%**	**98.5%**

For the **non-letterbox models**, the comparison is particularly useful:

- **Validation:** both achieved **98.38%**
- **Test:** no-dropout achieved **98.5%**, compared with **97.5%** with dropout.
- Therefore, removing dropout improved test accuracy by **1.0 percentage point**.

38.1.1 What does this tell us?

The interesting part is that **validation accuracy alone would suggest that dropout made no difference**:

> 98.38% vs. 98.38%

But the test results reveal a difference:

> **97.5% → 98.5%**

So in this experiment, the **no-dropout model generalized better to the test set**.

This is actually a good example of why we shouldn't choose between models based only on a tiny difference in validation accuracy. The two models looked identical according to their best validation accuracy, but their final test performance differed by **1 percentage point**.

38.1.2 Effect of Dropout

39 Final takeaway and final model selection:

> for my final model, the **non-letterbox + no-dropout ResNet18** is the stronger choice based on the results you have: **98.38% validation accuracy and 98.5% test accuracy**.

---

40 Experiment 14 --- OOD Detection with Energy and Mahalanobis Distance

40.1 Objective

The trained ResNet18 model is designed to classify images into eight known vehicle classes:

`ambulance`, `autobus`, `kamyun`, `kamyunet`, `minibus`, `savari`, `taxi`, and `vanet`.

Because the classifier is trained only on these eight classes, an unseen vehicle such as `neysan` is still forced into one of the known classes. Therefore, a high softmax confidence does not necessarily mean that an image belongs to the known distribution.

This experiment evaluates two Out-of-Distribution (OOD) detection methods:

1. **Energy Score**
2. **Mahalanobis Distance**

The objective is to determine whether an input is sufficiently different from the known training distribution to be treated as an unseen sample, without adding `neysan` as a ninth class to the classifier.

40.2 Model and Data Setup

The experiment uses an ImageNet-pretrained **ResNet18** model whose final fully connected layer is configured for eight classes.

The known classes are:

- `ambulance`
- `autobus`
- `kamyun`
- `kamyunet`
- `minibus`
- `savari`
- `taxi`
- `vanet`

The known dataset contains:

- **2,713 training images**
- **679 validation images**

The unseen `neysan` dataset is kept separate from the known training and validation data and contains:

- **586 unseen `neysan` images**

The preprocessing pipeline uses letterbox resizing to **224x224** followed by ImageNet normalization.

40.2.1 Checkpoint Used in This Run

This updated experiment uses the checkpoint:

```
resnet18\_full\_finetuning\_non\_letterbox\_best.pt
```

This is important because the OOD scores depend directly on the learned logits and feature representations of the selected model. Consequently, the numerical results of this updated run are different from an earlier Experiment 14 run that used a different checkpoint.

40.3 Energy Score

The Energy Score is computed from the model logits:

$$E(x) = -T \log \sum_i \exp\left(\frac{f_i(x)}{T}\right)$$

where:

- $f_i(x)$ is the logit for class i.
- T is the temperature parameter.
- $E(x)$ is the energy score.

In this experiment:

$$T = 1.0$$

Higher energy indicates that the input is less consistent with the known classification distribution and is therefore more likely to be OOD.

The Energy detector marks an image as OOD when:

$$E(x) > \text{Energy Threshold}$$

40.4 Mahalanobis Distance

The Mahalanobis detector operates on the 512-dimensional feature representation.

For each known class k, a mean feature vector μ_k is calculated. The covariance matrix is estimated using **Ledoit-Wolf shrinkage** to obtain a stable inverse covariance matrix.

The distance to class k is:

$$D_k(x) = \sqrt{(x - \mu_k)^T \Sigma^{-1} (x - \mu_k)}$$

The final Mahalanobis score is the distance to the closest known class:

$$D(x) = \min_k D_k(x)$$

A small distance means that the input is close to at least one known class, while a large distance indicates that the input is far from all known classes.

The Mahalanobis detector marks an image as OOD when:

$$D(x) > \text{Mahalanobis Threshold}$$

40.5 Threshold Selection

The OOD thresholds are selected **only from the known validation set**. The unseen `neysan` images are not used for threshold selection.

The thresholds are defined using the **95th percentile** of the validation scores.

For this updated run, the thresholds are:

Method OOD Threshold

Energy **-5.9057** Mahalanobis **20.1512**

The evaluation pipeline is therefore:

```
Known Training Data
        ↓
Extract 512{-}D Features
        ↓
Estimate Class Means and Covariance
        ↓
Known Validation Data
        ↓
Select 95th{-}Percentile Thresholds
        ↓
Unseen Neysan Data
        ↓
Final OOD Evaluation
```

This prevents the unseen `neysan` images from influencing threshold selection.

40.6 OOD Detection on Neysan

The two OOD detectors were applied independently to all **586 unseen `neysan` images**.

40.6.1 Energy Detection

The Energy detector identified:

- **35 / 586 images**
- **5.97%**

as OOD.

Therefore:

Energy OOD detection rate = 5.97%

40.6.2 Mahalanobis Detection

The Mahalanobis detector identified:

- **165 / 586 images**
- **28.16%**

as OOD.

Therefore:

Mahalanobis OOD detection rate = 28.16%

40.6.3 Combined Detection

The combined detector marks an image as OOD when **either Energy or Mahalanobis** exceeds its threshold:

The combined detector identified:

- **185 / 586 images**
- **31.57%**

as OOD.

The results are:

OOD Detection Method Neysan Detected as OOD Number

Energy **5.97%** 35 Mahalanobis **28.16%** 165 **Combined 31.57%** 185

The combined method detects more unseen samples than either individual detector. In this updated run, Mahalanobis provides the larger contribution to OOD detection.

40.7 Needs Review and Unknown Detection

A second decision layer is used to distinguish confidently detected unknown samples from samples that should be manually reviewed.

The decision rule is:

1. If **both Energy and Mahalanobis** identify the image as OOD → `unknown`
2. If **either one** identifies the image as OOD → `needs_review`
3. If neither detector identifies the image as OOD but softmax confidence is below **70%** → `needs_review`
4. Otherwise → `known_failure`

The resulting classification of the 586 unseen `neysan` images is:

Status Number of Images Percentage

Unknown 15 2.56% Needs review 170 29.01% OOD missed / known failure 401 68.43% Total 586 100%

Therefore, the combined OOD detector provides a screening mechanism, but a substantial portion of the unseen `neysan` images are still accepted as known samples.

40.8 Error Analysis

The 401 `neysan` images that were missed by the combined OOD detector were analyzed according to their predicted class.

All 401 missed images were classified as:

```
vanet
```

The detailed result is:

Predicted Count Mean Minimum Maximum Class Confidence Confidence Confidence

---

`vanet` **401 99.95% 97.08% 100.00%**

This is an important failure mode.

The model is not simply uncertain about the unseen `neysan` images. Instead, the missed images are classified as `vanet` with extremely high confidence.

This means that a simple confidence threshold cannot reliably solve the OOD problem for this unseen class.

The result suggests that many `neysan` images occupy a region of the learned feature space that is sufficiently close to the `vanet` representation to be accepted as in-distribution.

40.9 Discussion

The updated experiment again demonstrates the difference between ordinary classification confidence and OOD detection.

A ResNet18 classifier trained on eight classes will always produce one of those eight classes, even for an unseen `neysan` image. Therefore, softmax confidence alone cannot determine whether the image belongs to the known distribution.

In the updated experiment:

- Energy detects **5.97%** of the unseen `neysan` images.
- Mahalanobis detects **28.16%**.
- The combined detector detects **31.57%**.

Mahalanobis performs substantially better than Energy in this run. Combining the two methods improves the detection rate compared with either individual method, but the improvement remains limited.

The most significant failure is that **401 of the 586 unseen images (68.43%) are still missed**.

All 401 missed images are classified as `vanet`, with a mean confidence of **99.95%**. This indicates a strong visual and feature-space similarity between the unseen `neysan` images and the learned `vanet` representation.

40.10 Conclusion

This updated Experiment 14 evaluated OOD detection for the eight-class ResNet18 vehicle classifier using Energy Score and Mahalanobis Distance.

The thresholds were selected exclusively from the known validation set using the 95th percentile:

- **Energy threshold:** -5.9057
- **Mahalanobis threshold:** 20.1512

The main results are:

- **Energy:** 35 / 586 = **5.97%** detected as OOD.
- **Mahalanobis:** 165 / 586 = **28.16%** detected as OOD.
- **Combined Energy + Mahalanobis:** 185 / 586 = **31.57%** detected as OOD.
- **Unknown:** 15 images = **2.56%**.
- **Needs review:** 170 images = **29.01%**.
- **OOD missed / known failure:** 401 images = **68.43%**.
- All 401 missed images were predicted as **`vanet`**.
- The mean confidence for these missed images was **99.95%**.

Therefore, the combined OOD mechanism can provide a useful screening layer around the eight-class classifier, but in this updated run it is **not sufficient to reliably detect the unseen `neysan` class**.

The dominant failure mode is the strong similarity between `neysan` and `vanet` in the learned representation. Consequently, the OOD detector should be interpreted as a **screening and review mechanism rather than a guaranteed unseen-class detector**.

For future improvement, the `vanet--neysan` confusion should be investigated specifically using feature-space visualization, Grad-CAM/error analysis, and potentially additional OOD or representation-learning methods.

40.10.1 Result.json

#	Experiment key	Best epoch	Best val accuracy	Best val loss	Seed	Key configuration
1	baseline_cnn	9	78.94%	1.1164	42	SimpleCNN, no dropout, no GAP
2	augmentation	10	77.91%	0.7244	42	HFlip + Rotation(10) + ColorJitter(0.2)
3	dropout_02	9	75.55%	1.0750	42	dropout = 0.2
4	dropout_03	9	77.03%	0.9326	42	dropout = 0.3
5	dropout_05	10	74.96%	0.8809	42	dropout = 0.5
6	avg_pooling	5	72.45%	0.9533	42	MaxPool2d → AvgPool2d
7	weight_decay_0	10	76.14%	1.3452	42	AdamW, wd = 0
8	weight_decay_1e4	10	78.20%	1.1418	42	AdamW, wd = 1e-4
9	scheduler_reduce_on_plateau	8	78.65%	1.0207	42	ReduceLROnPlateau (factor 0.5, patience 2)
10	balanced_batches_standard_shuffle	3	73.64%	0.9005	42	Simulated imbalance (30% of ambulance/autobus/kamyun), shuffle=True
11	balanced_batches_sampler	7	66.86%	1.6913	42	Same imbalance, BalancedBatchSampler
12	bce_loss	5	78.49%	0.1776*	42	BCEWithLogitsLoss
13	gap	22	58.47%	1.1791	42	Global Average Pooling, up to 50 epochs
14	resnet18_feature_extraction	11	86.89%	0.4203	42	ResNet18, frozen backbone, `fc` only
15	resnet18_finetuning	12	93.96%	0.2703	42	`layer4` (lr 1e-4) + `fc` (lr 1e-3)
16	resnet18_full_finetuning	3	96.02%	0.1326	42	All layers, backbone lr 1e-4, head lr 1e-3
17	resnet18_full_finetuning_non_normalized	8	96.32%	0.1379	42	Same as #16, no ImageNet normalization
18	resnet18_full_finetuning_letterbox_no_dropout	20	98.09%	0.0863	7	Letterbox + HFlip + Rotation(10), head lr 1e-4, factor 0.03, patience 75
19	resnet18_full_finetuning_non_letterbox_no_dropout	122	**98.38%**	0.0887	7	Same as #18, without letterbox
20	resnet18_full_finetuning_letterbox_dropout	37	97.79%	0.1099	7	Letterbox + HFlip + Rotation(10), dropout (p = 0.3), head lr 1e-4, factor 0.03, patience 10
21	resnet18_full_finetuning_non_letterbox_dropout	70	98.38%	0.0786	7	HFlip + Rotation(10), dropout (p = 0.3), head lr 1e-4, factor 0.03, patience 10

* The BCE loss is not comparable to the cross-entropy losses in the other rows.